BACHELOR'S THESIS (TFG) · UC3M

Learning from every flight

Script for the full-pipeline presentation (16 slides, about 13 minutes)

[Your name] · Supervisor: [name] · September 2026

The thread

The talk starts with the reinforcement learning (TAILOR) as the core idea. It then walks through the eight-step pipeline that runs after every uploaded flight. Each supporting paper is introduced at the step where it is needed. The strip of step labels at the top of slides 6–13 shows where you are; the two RL steps are in teal.

#	Slide	Paper	Time
1	Title	—	0:20
2	**TAILOR: the core**	**TAILOR (NeurIPS 2023)**	**1:20**
3	**Adapted to one plant**	**TAILOR (my adaptation)**	**1:10**
4	**Worked example**	—	**1:00**
5	The full pipeline	—	0:40
6	Step 1 · Upload and tiles	—	0:50
7	Step 2 · Normalize	LUF-net (Plant Phenomics 2026)	0:50
8	Step 3 · Detect	—	0:45
9	**Step 4 · Select (RL)**	**TAILOR**	**0:50**
10	**Step 5 · Reward (RL)**	**TAILOR, adapted**	**0:50**
11	Step 6 · Train	Simple Supervision (arXiv 2026)	0:55
12	Step 7 · Retrain	On Warm-Starting (NeurIPS 2020)	0:50
13	Step 8 · Evaluate	—	0:45
14	Before vs. now	—	0:40
15	What is proven vs. to test	—	0:40
16	Simple signals win + questions	—	0:30

About 1,450 spoken words: 10–11 minutes at 140 words per minute, and about 13 minutes with pauses and pointing at the slides, which is what the slide timings allow for. The RL parts (slides 2–4, 9–10) take about 5 minutes. **For a 10-minute version**, skip slide 4 (worked example) and slide 8 (detect), and shorten slide 14 to one sentence.

Script

Slide 1 · Learning from every flight

0:00 – 0:20

On screen: title slide

> Hi. My TFG extends CARLA, which detects cardilla in drone video. Today I'll show a pipeline that lets the detector keep learning from every new flight, with as little labeling as possible. At its centre is a reinforcement-learning bandit, so I'll start there, and then walk through the whole pipeline.

Slide 2 · TAILOR: the core

0:20 – 1:40

On screen: how it works; 9 of 10 and = random

> After every flight, someone has to decide which few tiles a person should label. There are many ways to choose: the tiles the detector is least sure about, the ones most different from what we've labeled, and so on. The problem is that which one works best changes from dataset to dataset, and you can't know in advance.
>
> TAILOR, from NeurIPS 2023, doesn't choose one. It treats each strategy as an arm of a multi-armed bandit, the simplest form of reinforcement learning. For each labeling slot, Thompson sampling picks an arm, and that arm picks one example. After the person labels, each arm is rewarded from the labels it produced: examples of rare classes are worth more.
>
> Two results matter. On nine of ten datasets it was as good as the best single strategy, without knowing which one that was. And an older bandit rewarded by model accuracy did no better than random. **That's why the reward comes from the labels, not from retraining**, which also makes it cheap: one round per flight, no extra training.
>
> The limit: it was only tested on classification.

Slide 3 · Adapted to one plant

1:40 – 2:50

On screen: detector boxes → match with labels → yes/no facts

> But CARLA isn't classification, and it has only one class. TAILOR's reward values rare classes, so with one class it has nothing to balance. **My proposal is to reward rare cases instead.**
>
> When the bandit picks a tile, I save the detector's boxes for it. After the person labels the tile, I match the two by overlap, the same rule mAP uses. A labeled plant with no matching box is a miss. A box with no matching plant is a false positive. Together with whether the tile has cardilla and its altitude and light condition, that gives a small yes/no vector per tile, and TAILOR's multi-label version works on it unchanged.
>
> In practice it's almost free: if the tile is pre-filled with the detector's boxes in Roboflow, every box the person deletes is a false positive and every box they add is a miss.

Slide 4 · Worked example

2:50 – 3:50

On screen: table of arms, flight 1 vs. flight 2

> Here's what that looks like, with made-up numbers and fifty tiles per flight. On the first flight TAILOR knows nothing, so it spreads the slots evenly. The every-three-seconds arm mostly returns empty grass. Least-confident and flicker return misses and false positives, so they're rewarded.
>
> On the second flight they get most of the slots, while the others still get a few in case they improve.
>
> And the focus keeps moving. Once misses are common, they're worth less, and the reward shifts to whatever is still missing, for example overcast flights at forty metres.

Slide 5 · The full pipeline

3:50 – 4:30

On screen: eight step cards; steps 4 and 5 in teal

> That's the core. **Now the whole pipeline around it.** Every uploaded flight runs these eight steps once. Steps four and five are the reinforcement learning we just saw. The other steps make it work, and the other three papers come in at steps two, six and seven. I'll go through them in order.

Slide 6 · Step 1 · Upload

4:30 – 5:20

On screen: what a tile is; 8 tiles and ≈ 170,000

> Step one: a flight arrives as a video plus DJI's telemetry file, which logs altitude, GPS, ISO and shutter for every frame.
>
> Frames are cut into tiles: 640 by 640 pixel crops, eight per frame with some overlap, exactly as the repo's training script does. At twenty metres a tile covers roughly twelve by twelve metres of ground.
>
> For scale: the old rule labeled one frame every three seconds, about nineteen hundred tiles per flight. The whole video has around a hundred and seventy thousand candidate tiles. The bandit will choose a few dozen of them.

Slide 7 · Step 2 · Normalize (LUF-net)

5:20 – 6:10

On screen: 9.3% ⟶ 3.9% and 4.95%

> Step two, and the first supporting paper: LUF-net, Plant Phenomics 2026. Drone colours shift with the sun, the clouds and the camera's automatic exposure, so the same field looks different on different days. LUF-net corrects colour with a small network that is told the camera settings. Colour error drops from 9.3 percent with the image alone to 3.9. With only the exposure settings, which DJI already records, it's still about 5 percent.
>
> The limit: it measures colour, not detection, and training needs a colour chart. So it's a hypothesis to test against simple colour augmentation.

Slide 8 · Step 3 · Detect

6:10 – 6:55

On screen: 6 vs. 8 and 1/3 black

> Step three: the detector runs on every tile. Today that's the YOLO-nano model from the previous thesis.
>
> One thing to fix first: in the repo, training uses eight overlapping tiles per frame, but inference uses six non-overlapping ones, with the bottom row partly black padding. They should match, so the tiles the bandit picks look like what the detector actually sees.
>
> The boxes and confidences are saved, because three of the arms and the reward all need them.

Slide 9 · Step 4 · Select (RL)

6:55 – 7:45

On screen: the seven arms; k ≈ 25–100

> Step four is TAILOR in place. The bandit has seven arms, and **the first one is the old every-three-seconds rule.** The others use randomness, the detector's confidence, novelty, plants that flicker between frames, disagreement with T-Rex2, and rare conditions.
>
> For each of k slots, maybe twenty-five to a hundred per flight, Thompson sampling picks an arm and the arm picks a tile. One extra rule: never two tiles from the same spot, because neighbouring tiles and frames overlap so much. Then the person labels them in Roboflow, starting from the detector's boxes.

Slide 10 · Step 5 · Reward (RL) 7:45 – 8:35

On screen: worth table; “no retrain” and × 0.9

> Step five: the reward. Each fact is worth one over the number of times we’ve seen it so far, which is TAILOR’s own weighting. So a tile from a condition we’ve only seen three times is worth far more than yet another tile with cardilla.
>
> Two properties make this practical. There’s no retraining to compute it, because it comes straight from the labels. And older rounds count 0.9 times as much, so the bandit can follow flights that change through the season.

Slide 11 · Step 6 · Train (Simple Supervision) 8:35 – 9:30

On screen: +1.7 to +18.3 and −13.6

> Step six, and the second paper: Simple Supervision Is Hard to Beat, a 2026 preprint. Once the tiles are labeled, how do we train? The detector labels the rest of the flight itself, with pseudo-labels, and the new real labels are simply added as ordinary examples. They call it RTSM, and it gained 1.7 to 18.3 AP50 in every setting. Ten cleverer methods that used the labels to fix the pseudo-labels didn’t reliably help; one lost 13.6 points.
>
> **Importantly, the authors chose the labeled images at random on purpose. That’s exactly the part TAILOR replaces**, and it gives the fair baseline: random tiles, same recipe.

Slide 12 · Step 7 · Retrain (On Warm-Starting) 9:30 – 10:20

On screen: 56.2 ⟶ 51.7% and “same accuracy”

> Step seven, third paper: On Warm-Starting, NeurIPS 2020. The loop only works if we can retrain after every flight. From scratch is too slow on one GPU. Continuing from the last weights is faster, but it generalizes worse: on CIFAR-10, 56 versus 52 percent.
>
> Their fix is Shrink and Perturb: multiply the weights by about 0.6, add a little noise, then fine-tune. It matched training from scratch and was much faster. The caveat: no data augmentation in their experiments, and YOLO uses a lot, so I’ll measure it.

Slide 13 · Step 8 · Evaluate 10:20 – 11:05

On screen: same budget; mAP vs. labels

> Step eight closes the round. **The test set must not come from the bandit**: it picks hard tiles on purpose, so every model would look worse than it really is. Instead, test tiles come from flights the loop never saw, sampled every three seconds or at random, and split by flight and by condition.
>
> Every method gets the same labeling budget, and the headline figure is accuracy against the number of labeled tiles. Then the next flight arrives, and the loop starts again.

Slide 14 · Before vs. now 11:05 – 11:45

On screen: two columns: every 3 s vs. the bandit

> So how does this compare with what CARLA did before? The every-three-seconds rule doesn’t disappear. It becomes one arm. If it really is the best rule, the bandit learns to pick it; if not, labels move to the tiles that teach the detector most. The price is a retrain per flight and some labels spent exploring. And the old rule keeps a job: it provides the unbiased test set.

Slide 15 · What is proven vs. to test

11:45 – 12:25

On screen: evidence table, RL rows highlighted

> To be honest about the evidence: every paper shows something in its own setting, and none was tested on a single-class drone detector. The two highlighted rows, the bandit and its reward, are the core claims the thesis has to test. The first experiment is a simulation: fully label one or two flights, hide the labels, and reveal them only when a method asks for a tile.

Slide 16 · Simple signals win

12:25 – 12:55

On screen: three lesson cards and two questions

> To close: three of these papers make the same point. Simple, well-chosen signals beat elaborate ones. So I'll always report the simple baselines next to the RL. Two questions for you: is a bandit enough RL for the thesis, and can I fully label one or two flights to simulate the loop? Thank you.

Linking sentences

These lines hold the talk together; keep them even when shortening.

- **Slide 5:** "That's the core. Now the whole pipeline around it." (Switch from TAILOR to the pipeline.)
- **Slide 9:** "The first one is the old every-three-seconds rule." (The new method contains the old one.)
- **Slide 11:** "The authors chose the labeled images at random on purpose. That's exactly the part TAILOR replaces." (Simple Supervision and TAILOR answer complementary questions.)
- **Slide 13:** "The test set must not come from the bandit." (Why evaluation stays unbiased.)
- **Slide 16:** "Simple signals win." (The lesson three of the papers share.)

Backup answers for questions

If asked…	Answer
How does TAILOR know the detector missed a plant?	It doesn't by itself. The pipeline saves the detector's boxes when a tile is picked and matches them with the person's labels by overlap (IoU ≥ 0.5, as in mAP50): a label with no box is a miss, a box with no label is a false positive. With pre-filled tiles, the person's deletions and additions give the same answer.
What exactly is a tile?	A 640 × 640 px crop of a 1920 × 1080 frame; 8 per frame with 160–320 px overlap, as in the repo's `dataset_tiling.py`. About 12 × 12 m of ground at 20 m altitude (assuming an 84° field of view, to be checked).
Will the bandit really beat every 3 s?	Not guaranteed. TAILOR matched the best single strategy on 9 of 10 datasets, but with far more labels per round. The fair test is the same number of labeled tiles per method, the same training recipe, and mAP50 against labels on held-out flights.
Why a bandit and not a "real" RL policy?	A learned policy (e.g. MGRAL, ICASSP 2026, policy gradient with an mAP reward) needs many retrains per update. With one GPU and one round per flight, a bandit is what fits. A learned policy is a possible extension.
Why not use the mAP gain as the reward?	It needs a retrain per reward and is noisy between seeds. In TAILOR, the accuracy-rewarded bandit (ALBL) did no better than random. mAP is kept for evaluation, and comparing both rewards is itself an experiment.
TAILOR used thousands of labels per round; you'll have tens.	True, it is a real risk. I'll test it first in simulation on fully labeled flights, where many rounds can be run cheaply.
Does LUF-net work on compressed video?	Untested; the paper used still images. It will be compared against plain colour augmentation, which is the cheap baseline.
Does Shrink & Perturb still matter with YOLO's augmentation?	Unknown; the paper had no augmentation. I'll compare warm start, Shrink & Perturb and restarting from COCO weights, in accuracy and GPU time.

Delivery tips

- Spend the most care on slides 2–4: if the audience understands the bandit, the rest of the pipeline follows.
- On slides 6–13, point at the step strip at the top as you say "Step one", "Step two"…, and on steps 4–5 point out the teal: that is the RL.
- On each paper slide, say only the two numbers on the right; the bullets are there for reading later.
- Say "illustrative" on slides 4 and 10: those numbers are made up to show the mechanics.